

CST8324 Project Winter 2026

Project outline

The purpose of the program is to manage servers.

- Server data is stored in a CSV file.
- *Server data format:* hostname, domain, ipv4, port, location
Example: `www,happy.lab,172.16.30.101,443,wt306`
- The following actions can be performed for server management: list, search, add, delete.
- The updated server list is written to a JSON file.

Project objective

- Build a program to manipulate file data that meets user requirements.
- Write a maintainable program file that respects Python style guide.
- Test the program based on a comprehensive test plan.
- Manage different code versions with a version control system.

Program name

`inventory_servers.py`

Project overview

Program description

The program is menu driven and executes until the user quits.

Server data is read in from a CSV file as a list of dictionaries.

Based on the user-selected menu option, the corresponding tasks can be performed:

- List all servers.
- List all servers based on a user-selected attribute (a dictionary key).
- Search all servers based on a user-selected search criteria (a dictionary value).
- Add a server with user-provided server values for each key.
- Delete a server based on the FQDN.

Once the user quits server management tasks, the updated list of servers is written to a JSON file.

Python style guide (back by popular demand!)

The following style guide rules are expected:

- Commenting: program header block comment, code section comments

- Naming style guide for variables and functions
- Formatting: spacing & indentation for readability

Python programming constructs

The following Python programming constructs are implemented in the program:

- Apply basic expressions.
- Implement flow control.
- Decompose with functions.
- Process lists and dictionaries.
- Read from and write to file.
- Handle file exceptions.
- Read CSV data.
- Write JSON data.

Functions

Function list:

- Functions that are provided:
 - `show_menu()`
 - `get_menu_option()`
 - `get_search_string()`
 - `get_server_info()`
 - `find_fqdn()`
 - `list_servers()`
 - `delete_server()`
 - `read_csv()`
- Functions that are NOT provided:
 - `get_keys()`
 - `search_server()`
 - `add_server()`
 - `write_json()`

Project details

Function details

Functions that are provided:

- `show_menu()`
 - Function purpose: display menu options
 - Function parameters: none
 - Function return: none

- Function body: display menu
- `get_menu_option()`
 - Function purpose: get menu option
 - Function parameters: prompt
 - Function return: menu option
 - Function body: prompt user for menu option
- `get_search_string()`
 - Function purpose: get a search string
 - Function parameters: prompt
 - Function return: search string
 - Function body: prompt user for search string
- `get_server_info()`
 - Function purpose: get values for a new server
 - Function parameters: prompt, list of keys
 - Function return: key-value pairs for a server
 - Function body:
 - for each key, prompt user for value
 - create a server with all key-value pairs
- `find_fqdn()`
 - Function purpose: find server based on fqdn
 - Function parameters: list of servers, fqdn
 - Function return: matching server or False
 - Function body:
 - for each server, compare fqdn with hostname and domain values in list of servers
- `list_servers()`
 - Function purpose: list all servers, optionally by a given server attribute
 - Function parameters: list of servers, optional attribute
 - Function return: none
 - Function body:
 - If an attribute is provided, display the corresponding value for all servers
 - Otherwise, display all attribute values of all servers
- `delete_server()`
 - Function purpose: delete a server from the server list
 - Function parameters: server list, server to delete
 - Function return: updated server list
 - Function body: delete server

- `read_csv()`
 - Function purpose: read in data from a CSV file
 - Function parameters: CSV input file
 - Function return: CSV data formatted as list of dictionaries
 - Function body:
 - Open file in read access mode with exception handling
 - For each CSV entry, convert data to a dictionary and append to list

Functions that are NOT provided:

- `get_keys()`
 - Function purpose: Extract keys from a list of dictionaries.
 - Function parameters: server list (*alternatively*: a single server)
 - Function return: list of dictionary keys
 - Function body:
 - Extract the keys from a server entry.
Note: Select the first dictionary in the list, which is the first list element.
 - Return the keys.
Note: Apply the relevant dictionary method.
 - Convert returned keys to a list.
Note: Use the `list` type conversion function.
- `search_server()`
 - Function purpose: To find servers that match an attribute value
 - Function parameters: server list, search string
 - Function return: list of matching servers
 - Function body: Add every server that matches the search string to a list of matching servers.
- `add_server()`
 - Function purpose: Add a server to the server list
 - Function parameters: server list, server attribute values
 - Function return: updated server list
 - Function body: Add a new server to the server list
- `write_json()`
 - Function purpose: write data to JSON file
 - Function parameters: JSON destination file, server data
 - Function return: none
 - Function body:
 - Open destination file with exception handling.

- Handle the “Exception” object; additional exception handlers are optional.
- The exception handler displays a message; do not exit - the file is not essential to the program.
- Write the server data to the file in JSON format.

Function calls per server management task

- To list servers, the following functions are called:
 - `list_servers()`
 - `get_search_string()`
 - `get_keys()`
- To search for servers, the following functions are called:
 - `search_server()`
 - `get_search_string()`
- To add a server, the following functions are called:
 - `add_server()`
 - `get_keys()`
 - `get_server_info()`
- To delete a server, the following functions are called:
 - `delete_server()`
 - `get_search_string()`
 - `find_fqdn()`

Test plan

Below, find a list of test cases to be included in the test plan:

- Exception handlers for file I/O
 - Invalid menu option
 - Quit menu option
 - List menu option with default values
 - List menu option using user-selected key (test each key)
 - List menu option using user-selected invalid key
 - Search menu option using user-selected attribute with single match
 - Search menu option using user-selected attribute with multiple matches
 - Search menu option using user-selected invalid attribute
 - Add server menu option providing all attributes
 - Add server menu option providing incomplete server attributes
- Note: This program version does not include server data validation, such as mandatory attributes (hostname & domain name) or data validation (such as IP format validation)*

- Delete server menu option using fqdn
- Delete server menu option using invalid fqdn
- Iteration: quit immediately
- Iteration: quit after one run
- Iteration: quit after multiple runs
- All csv server data read in as list of dictionaries
- Json data created with updated server data

Approach

- Download and carefully review the project documentation
 - Download & inspect the skeleton program.
 - Download & inspect the sample data CSV file.
 - Download & inspect the sample output.
- Read and comment the code:
 - Add the program header comment.
 - Add a section comment for each code section.
Note: Add a comment above each code section where a '#' is placed.
 - Add additional comments where they are helpful for your understanding of the program code.
- Build the test plan based on the test cases provided.
- Write & debug code, one section at a time.
 - Complete the main code.
 - Complete the "Search server" option.
 - Complete the "Add server" option.
 - Complete the missing functions.
 - `get_keys()`
 - `search_server()`
 - `add_server()`
 - `write_json()`
- Test the code and complete the test plan.
- Prepare the submission based on the submission details document.
 - The following elements are part of the submission:
 - Program file
 - Test plan
 - Sample output
 - JSON file
 - Version control history

- Consult the project rubric to ensure that your project follows the set guidelines.